

TAREA 02
PARADIGMAS DE LA
PROGRAMACIÓN
NRC 13046

Integrante:

Martín Cerpa Díaz, 21.629.439-4

Docente: Juan Calderon

Fecha de entrega: 28 de Mayo de
2026

Índice

Índice.....	2
INTRODUCCIÓN.....	2
DESARROLLO.....	3
1. Descripción de la solución.....	3
2. Organización del proyecto.....	3
2.1 Paquete backend.....	4
2.2 Paquete frontend.....	4
3. Clases principales del sistema.....	4
3.1 Clase Main.....	4
3.2 Clase Ejercicio.....	4
3.3 Enum TipoEjercicio.....	5
3.4 Enum IntensidadEjercicio.....	5
3.5 Clase Rutina.....	5
3.6 Clase GestorAplicacion.....	6
3.7 Clase GeneradorRutina.....	6
3.8 Clase CargadorEjercicios.....	7
3.9 Clase CargadorBD.....	7
3.10 Clase CreadorBD.....	7
3.11 Interface FuenteDatos.....	7
3.12 Interface ObservadorSistema.....	8
4. Ventanas del sistema.....	8
4.1 VentanaPrincipal.....	8
4.2 VentanaCarga.....	8
4.3 VentanaGeneracion.....	9
4.4 VentanaRevision.....	9
4.5 VentanaResumen.....	9
4.6 VentanaRutinasExistentes.....	10
5. Decisiones de diseño.....	10
5.1 Uso de Java Swing.....	10
5.2 Separación backend y frontend.....	10
5.3 Uso de Vector.....	10
5.4 Carga desde CSV y SQLite.....	11
5.5 Validaciones y excepciones.....	11
5.6 Restricción de semanas consecutivas.....	11
5.7 Historial de rutinas.....	11
5.8 Notificación-suscripción.....	11
6. Diagrama.....	12
7. Flujo general del sistema.....	13
8. Supuestos considerados.....	13
9. Conclusión.....	13

INTRODUCCIÓN

El presente informe describe el desarrollo de un sistema de gestión de rutinas de entrenamiento personalizadas, realizado para la asignatura Paradigmas de Programación. El objetivo principal del proyecto fue construir una aplicación en Java con interfaz gráfica Swing, capaz de cargar ejercicios desde un archivo o base de datos, generar rutinas según ciertos criterios y permitir la revisión de los ejercicios seleccionados.

El problema planteado se relaciona con la necesidad de administrar rutinas para clientes de un centro deportivo. Cada rutina se compone de ejercicios cardiovasculares y de fuerza, los cuales poseen un nivel de intensidad, tiempo estimado de ejecución, descripción y registro de la última semana en que fueron utilizados.

Para resolver el problema se aplicaron conceptos de programación orientada a objetos, modularización, manejo de eventos, validaciones y separación entre la lógica del sistema y la interfaz gráfica. La aplicación fue organizada en dos paquetes principales: backend, encargado de la lógica y manejo de datos, y frontend, encargado de las ventanas y la interacción con el usuario.

El informe se estructura en una descripción general de la solución, explicación de las clases utilizadas, relaciones entre ellas, decisiones de diseño, diagrama de clases y conclusión del trabajo realizado.

DESARROLLO

1. Descripción de la solución

El sistema desarrollado permite cargar ejercicios desde dos fuentes de datos: archivo CSV o base de datos SQLite. Una vez cargados, los ejercicios se mantienen en memoria durante la ejecución del programa. Posteriormente, el usuario puede generar una rutina indicando el nombre del cliente, la cantidad de ejercicios cardiovasculares, la cantidad de ejercicios de fuerza, la intensidad requerida para cada tipo y la semana actual.

La rutina se genera considerando los ejercicios disponibles, el tipo de ejercicio, el nivel de intensidad solicitado y la restricción de no repetir ejercicios usados en la semana inmediatamente anterior. Si no existen suficientes ejercicios disponibles para cumplir la solicitud, el sistema muestra un mensaje de error.

Después de generar la rutina, el usuario puede revisar los ejercicios uno por uno. La ventana de revisión permite avanzar, retroceder y llegar al resumen final. En el primer ejercicio, el botón para volver se deshabilita. En el último ejercicio, el botón de siguiente cambia a “Resumen de la rutina”.

Además, se agregó una funcionalidad para revisar rutinas existentes generadas durante la misma ejecución del programa. Estas rutinas se almacenan en memoria y pueden ser consultadas desde la ventana principal o desde la ventana de revisión.

2. Organización del proyecto

El proyecto fue organizado en la siguiente estructura:

```
Programa/
├── datos/
│   └── ejercicios.csv
├── lib/
│   └── sqlite-jdbc.jar
├── src/
│   ├── backend/
│   ├── frontend/
│   └── Main.java
├── compilar.bat
├── ejecutar.bat
└── README.md
```

La separación en paquetes permite dividir claramente las responsabilidades del sistema.

2.1 Paquete backend

El paquete backend contiene la lógica del sistema. En este paquete se encuentran las clases relacionadas con los ejercicios, rutinas, carga de datos, generación de rutinas, validaciones y comunicación con observadores.

2.2 Paquete frontend

El paquete frontend contiene las ventanas de la interfaz gráfica. Estas ventanas permiten al usuario cargar ejercicios, generar rutinas, revisar ejercicios, ver resúmenes y consultar rutinas existentes.

3. Clases principales del sistema

3.1 Clase Main

La clase Main corresponde al punto de inicio del programa. Su función es crear y mostrar la ventana principal de la aplicación.

Esta clase no contiene lógica de negocio, ya que solamente inicia la ejecución del sistema.

3.2 Clase Ejercicio

La clase Ejercicio representa cada ejercicio disponible para formar parte de una rutina. Contiene los datos principales solicitados por el problema:

- Código identificador.
- Nombre del ejercicio.
- Tipo de ejercicio.
- Nivel de intensidad.
- Tiempo estimado.
- Descripción de ejecución.
- Última semana en que fue utilizado.

Esta clase permite manejar cada ejercicio como un objeto, facilitando el acceso a sus atributos mediante métodos getters y setters.

3.3 Enum TipoEjercicio

El enum TipoEjercicio define los tipos de ejercicios permitidos por el sistema. Los valores utilizados son:

- CARDIOVASCULAR
- FUERZA

El uso de un enum permite evitar errores de escritura y restringir los valores válidos dentro del programa.

3.4 Enum IntensidadEjercicio

El enum IntensidadEjercicio representa los niveles de intensidad permitidos:

- BASICO
- INTERMEDIO
- AVANZADO
- ALTO_RENDIMIENTO

Esta decisión permite trabajar con intensidades definidas y facilita las validaciones al momento de cargar datos o generar rutinas.

3.5 Clase Rutina

La clase Rutina representa una rutina generada para un cliente. Contiene un conjunto de ejercicios almacenados en memoria usando Vector.

Además, la rutina posee datos como:

- Identificador de rutina.
- Cliente.
- Semana.
- Lista de ejercicios.

También incluye métodos para calcular información de resumen, como:

- Cantidad total de ejercicios.
- Cantidad de ejercicios por tipo.
- Cantidad de ejercicios por intensidad.
- Tiempo total estimado.

3.6 Clase GestorAplicacion

La clase GestorAplicacion funciona como clase central del backend. Se encarga de coordinar las operaciones principales del sistema.

Sus responsabilidades son:

- Cargar ejercicios desde archivo o base de datos.
- Mantener los ejercicios en memoria.
- Generar rutinas.
- Guardar la rutina actual.
- Mantener un historial de rutinas generadas.
- Permitir seleccionar rutinas existentes.
- Notificar cambios al frontend mediante observadores.

Esta clase permite que las ventanas no trabajen directamente con todos los detalles internos del sistema, sino que se comuniquen con un gestor principal.

3.7 Clase GeneradorRutina

La clase GeneradorRutina contiene la lógica para construir una rutina. Para esto considera:

- Cantidad solicitada de ejercicios cardiovasculares.
- Cantidad solicitada de ejercicios de fuerza.
- Intensidad requerida para cada tipo.
- Semana actual.
- Ejercicios disponibles.
- Restricción de no repetir ejercicios en semanas consecutivas.

La selección de ejercicios se realiza filtrando los ejercicios disponibles según tipo, intensidad y última semana de uso. Si no hay suficientes ejercicios disponibles, se informa el problema al usuario.

3.8 Clase CargadorEjercicios

La clase CargadorEjercicios se encarga de leer ejercicios desde un archivo CSV. Esta clase valida:

- Existencia del archivo.
- Formato correcto.
- Información incompleta.
- Tipo de ejercicio válido.
- Intensidad válida.
- Tiempo estimado mayor a cero.
- Semana de uso mayor o igual a cero.

Si se detecta un problema, se lanza una excepción con un mensaje indicando el error.

3.9 Clase CargadorBD

La clase CargadorBD permite cargar ejercicios desde una base de datos SQLite. Su función es conectarse a la base de datos, leer los registros y convertirlos en objetos de tipo Ejercicio.

También valida que los datos obtenidos desde la base de datos sean correctos antes de incorporarlos al sistema.

3.10 Clase CreadorBD

La clase CreadorBD permite crear una base de datos de ejemplo con ejercicios. Esta funcionalidad facilita probar el programa sin tener que construir manualmente la base de datos.

3.11 Interface FuenteDatos

La interface FuenteDatos permite representar una fuente de datos de manera general. Gracias a esto, el sistema puede trabajar con distintas formas de carga, como CSV o base de datos, manteniendo una estructura más ordenada.

3.12 Interface ObservadorSistema

La interface ObservadorSistema se utiliza para implementar un mecanismo simple de notificación-suscripción. Las ventanas pueden actuar como observadores del backend y recibir mensajes cuando ocurre algún evento importante.

Por ejemplo:

- Cuando se cargan ejercicios correctamente.
- Cuando se genera una rutina.
- Cuando se selecciona una rutina existente.

Este mecanismo permite separar mejor la lógica del backend y la interfaz gráfica.

4. Ventanas del sistema

4.1 VentanaPrincipal

La VentanaPrincipal corresponde al menú inicial del sistema. Desde esta ventana el usuario puede acceder a las principales funciones:

- Cargar ejercicios.
- Crear base de datos.
- Ver rutinas existentes.
- Iniciar el flujo del sistema.

Esta ventana se mantiene como entrada principal de la aplicación.

4.2 VentanaCarga

La VentanaCarga permite seleccionar la fuente de datos desde donde se cargarán los ejercicios. Después de la carga, muestra información general como:

- Cantidad total de ejercicios.
- Tiempo total disponible.
- Cantidad de ejercicios por tipo.
- Cantidad de ejercicios por intensidad.

Luego permite continuar hacia la generación de rutina.

4.3 VentanaGeneracion

La VentanaGeneracion permite ingresar los datos necesarios para generar una rutina:

- Nombre del cliente.
- Cantidad de ejercicios cardiovasculares.
- Intensidad cardiovascular.
- Cantidad de ejercicios de fuerza.
- Intensidad de fuerza.
- Semana actual.

Esta ventana valida los datos ingresados antes de solicitar la generación de la rutina al backend.

4.4 VentanaRevision

La VentanaRevision permite revisar una rutina ejercicio por ejercicio. Muestra:

- Nombre del ejercicio.
- Tipo.
- Intensidad.
- Tiempo estimado.
- Descripción.

Además, mantiene el estado de navegación mediante una posición actual dentro de la rutina. El usuario puede avanzar, volver o acceder a rutinas existentes.

4.5 VentanaResumen

La VentanaResumen muestra el resumen final de una rutina. Incluye:

- Cantidad total de ejercicios.
- Cantidad de ejercicios cardiovasculares.
- Cantidad de ejercicios de fuerza.
- Cantidad por intensidad.
- Tiempo total estimado.

Esta ventana permite verificar de manera rápida la composición final de la rutina generada.

4.6 VentanaRutinasExistentes

La VentanaRutinasExistentes permite consultar las rutinas generadas durante la ejecución del programa. Muestra datos como:

- ID de la rutina.
- Cliente.
- Semana.
- Cantidad total de ejercicios.
- Tiempo total estimado.

Desde esta ventana se puede seleccionar una rutina anterior y revisarla nuevamente.

5. Decisiones de diseño

5.1 Uso de Java Swing

Se utilizó Swing porque el enunciado solicita una interfaz gráfica desarrollada en Java. Swing permite crear ventanas, botones, listas, tablas y cuadros de texto sin instalar librerías adicionales.

5.2 Separación backend y frontend

El proyecto se separó en dos paquetes principales:

- backend: contiene la lógica del sistema.
- frontend: contiene la interfaz gráfica.

Esta separación permite que el código sea más ordenado y facilita la mantención del sistema.

5.3 Uso de Vector

Los ejercicios y rutinas se mantienen en memoria usando Vector, ya que el enunciado indica que la información debe mantenerse durante la ejecución del programa. Esta estructura permite almacenar objetos de forma dinámica y recorrerlos fácilmente.

5.4 Carga desde CSV y SQLite

Se implementaron dos formas de carga: archivo CSV y base de datos SQLite. Esto entrega mayor flexibilidad al sistema y permite cumplir con la indicación de cargar ejercicios desde archivo o BD.

5.5 Validaciones y excepciones

El sistema valida errores comunes, como archivos inexistentes, formato incorrecto, datos incompletos o valores inválidos. Estas validaciones se manejan mediante excepciones, mostrando mensajes claros al usuario.

5.6 Restricción de semanas consecutivas

Para evitar repetir ejercicios en semanas consecutivas, cada ejercicio posee un atributo `ultimaSemanaUso`. Al generar una rutina, el sistema descarta ejercicios cuya última semana de uso corresponda a la semana inmediatamente anterior.

Por ejemplo, si la semana actual es 5, no se consideran disponibles los ejercicios usados en la semana 4.

5.7 Historial de rutinas

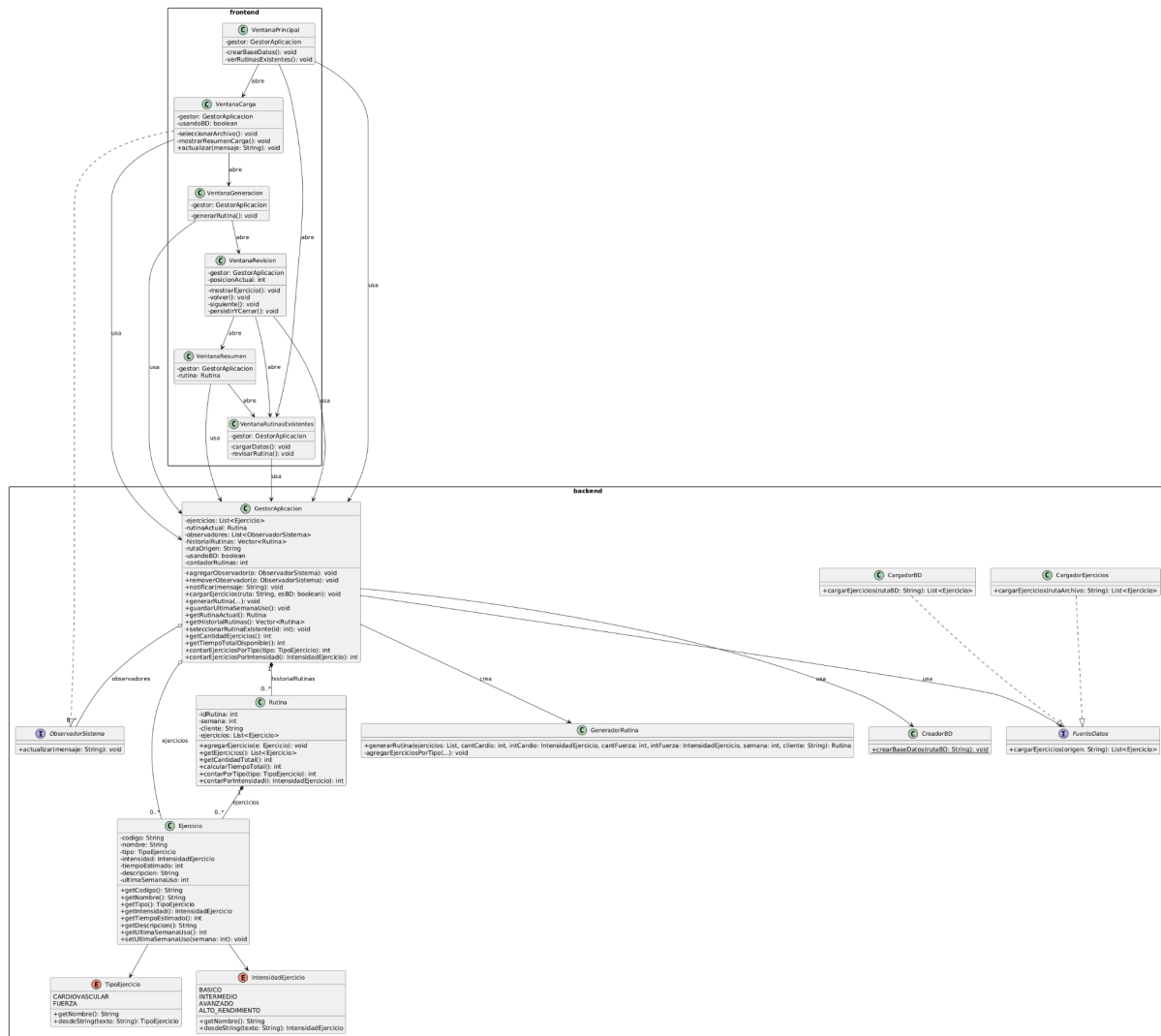
Se agregó un historial de rutinas generadas durante la ejecución del programa. Esto permite revisar rutinas existentes sin tener que generarlas nuevamente. El historial se mantiene en memoria, por lo que se pierde al cerrar la aplicación.

5.8 Notificación-suscripción

El sistema utiliza una interfaz de observador para comunicar eventos entre el backend y el frontend. De esta forma, el backend puede notificar a las ventanas cuando ocurre una acción relevante.

Esta decisión permite aplicar programación orientada a eventos y mantener una comunicación más ordenada entre las partes del sistema.

El siguiente diagrama representa de forma general las principales clases del sistema y sus relaciones.



7. Flujo general del sistema

El flujo principal del programa es el siguiente:

1. El usuario abre la aplicación.
2. El usuario carga ejercicios desde CSV o base de datos.
3. El sistema valida y almacena los ejercicios en memoria.
4. El usuario ingresa los datos para generar una rutina.
5. El backend filtra los ejercicios según tipo, intensidad y semana.
6. Se genera la rutina y se guarda en el historial.
7. El usuario revisa los ejercicios uno por uno.
8. Al final, el sistema muestra un resumen de la rutina.
9. El usuario puede volver a revisar rutinas generadas anteriormente.

8. Supuestos considerados

Durante el desarrollo se consideraron los siguientes supuestos:

- El archivo CSV debe usar punto y coma como separador.
- Los tipos válidos son CARDIOVASCULAR y FUERZA.
- Las intensidades válidas son BASICO, INTERMEDIO, AVANZADO y ALTO_RENDIMIENTO.
- La semana actual debe ser mayor a cero.
- El valor ultimaSemanaUso = 0 indica que el ejercicio no ha sido usado anteriormente.
- Las rutinas existentes se guardan solo durante la ejecución del programa.
- Si no hay suficientes ejercicios disponibles, no se genera una rutina incompleta.
- La ventana principal funciona como menú de acceso a las opciones del sistema.

9. Conclusión

El desarrollo de este proyecto permitió aplicar conceptos importantes de la programación orientada a objetos en Java, como la creación de clases, encapsulamiento, uso de enums, manejo de colecciones, separación de responsabilidades y comunicación entre objetos.

El sistema cumple con las funcionalidades principales solicitadas, ya que permite cargar ejercicios desde archivo o base de datos, generar rutinas personalizadas, revisar ejercicios individualmente y visualizar un resumen final. Además, se incorporó un historial de rutinas generadas durante la ejecución, lo que mejora la revisión de información dentro del programa.

Una de las decisiones más importantes fue separar el proyecto en los paquetes backend y frontend, ya que esto permitió organizar mejor la lógica del sistema y la interfaz gráfica. También fue relevante aplicar validaciones y manejo de excepciones para evitar errores al cargar datos o generar rutinas.

Como aprendizaje principal, el trabajo permitió comprender mejor cómo una aplicación de escritorio puede combinar interfaz gráfica, lógica de negocio, manejo de archivos, base de datos y eventos. Además, permitió reforzar la importancia de diseñar el sistema antes de programar, para que las clases tengan responsabilidades claras y el código sea más fácil de mantener.

En conclusión, el sistema desarrollado entrega una solución funcional al problema planteado y aplica los contenidos principales vistos en la asignatura, especialmente programación orientada a objetos, eventos, modularización y uso de Java Swing.